The 256 B – 1 KiB Slow Zone on MI250X xGMI with UCX 1.18 + Open MPI 5.0.7 + ROCm 6.4.1 (EESSI 2025.06)

TL;DR

- The dip is not a UCX tuning bug but a structural floor: between roughly 200 B and 1 KiB, every available D→D path in this stack — eager-over-`rocm_copy` with host-pinned staging, and rendezvous-over-`rocm_ipc` — pays a fixed per-transfer cost (HSA IPC handle create/attach for `rocm_ipc`, ~10 µs interface "latency" advertised by `rocm_copy`, plus a `hipMemcpy` / blit-kernel launch on each side) that is much larger than the few hundred nanoseconds needed to move 256 B over a 200 GB/s xGMI link, so bandwidth ≈ payload / fixed-cost stays in the 80–200 MB/s range until messages grow large enough to amortise it. AMD's own published `osu_bw` curve on the openucx wiki shows the identical dip (128 B → 61.11 MB/s, 256 B → 92.35 MB/s, 512 B → 160.11 MB/s, 1024 B → 167.99 MB/s). github +3
- No `UCX_RNDV_THRESH` value fixes the band, because moving the eager/rendezvous boundary just chooses *which* fixed-cost path pays the toll; the only knobs that change the curve meaningfully are `HSA_ENABLE_SDMA` (blit-kernel vs SDMA, which shifts the asymptote, not the floor), trusting `UCX_PROTO_ENABLE=y` (protov2, default since UCX 1.16) to keep `rocm_copy` eager all the way down, or — as HPE Cray MPICH does — keeping small GPU-GPU transfers on a pre-registered host-staged shared-memory path (`MPICH_GPU_EAGER_REGISTER_HOST_MEM=1`, `MPICH_GPU_IPC_THRESHOLD ≈ 1024 B`) so that they never touch `hsa_amd_ipc_memory_attach` at all. Readthedocs
- The slow zone is largely *path-invariant* on MI250X: intra-OAM (4-link, 200 GB/s peak), inter-package 2-link (100 GB/s) and 1-link (50 GB/s) GCD pairs all share the same `rocm_ipc`/`rocm_copy` software stack and the same HSA small-copy floor, so the *location* (256 B – 1 KiB) and *MB/s depth* of the dip should stay almost the same; what changes with topology is only the asymptote at large messages and the cliff position where the curve finally lifts off (around 8–16 KiB on quad-link, slightly later on 1-link and routed pairs). This is a mechanistic prediction consistent with — but not directly published in — Schieffer et al. ("Understanding Data Movement in AMD Multi-GPU Systems with Infinity Fabric," arXiv:2410.00801, SC-W '24) and De Sensi et al. ("Exploring GPU-to-GPU Communication: Insights into Supercomputer Interconnects," SC24, DOI:10.1109/SC41406.2024.00039), and should be confirmed empirically on the LUMI standard-g node. Lumi-supercomputer + 3

Key Findings

1. What is actually slow

The 256 B – 1 KiB band on intra-OAM `osu_bw` is the region where:

- the message is too large to be sent inline in a UCT short header — UCX `rocm_copy` advertises `put_short` and `get_short` but no

`am_short`/`am_bcopy`/`put_bcopy`/`get_bcopy`, so for GPU buffers UCP cannot complete a transfer with a single inline payload; *every* D→D message touches a real copy engine; and
- the message is too small to amortise the per-transfer setup of either `rocm_copy` (host-pinned-staging short/zcopy that pays a multi-µs interface latency per call in the UCT cost model) or `rocm_ipc` (put/get-zcopy that requires `hsa_amd_ipc_memory_create` on the sender and `hsa_amd_ipc_memory_attach` on the receiver, followed by `hsa_amd_memory_async_copy` between the two GCDs).

The `ucx_info -d` capability dump from a public UCX 1.10 + ROCm node (LLNL Corona, openucx issue #6207, same `iface_query` code paths as 1.18) advertises:

- `rocm_copy`: `latency: 10000 nsec, overhead: 0 nsec, bandwidth: 6911 MB/sec`, capabilities `put_short`, `put_zcopy`, `get_short`, `get_zcopy`. github
- `rocm_ipc`: `latency: 80 nsec, overhead: 400 nsec, bandwidth: 10240 MB/sec`, capabilities only `put_zcopy`, `get_zcopy`. github

The UCP protocol-selection layer uses these fields to estimate `time = overhead + latency + size / bandwidth` per candidate protocol and picks the minimum; for `rocm_ipc` the model says "~10 µs of setup, then >10 GB/s" so the bandwidth crossover is around 1–8 KiB, which is exactly where the curve lifts off in every published MI250X `osu_bw` dataset.

2. The dip is in AMD's own published curve

The openucx wiki page "Build and run ROCm UCX OpenMPI" — the canonical AMD recommendation for this stack — publishes an `osu_bw` run with `HSA_ENABLE_SDMA=0 -x UCX_RNDV_THRESH=256k` on inter-die GCDs 0–1 of an MI250/MI250X OAM. The verbatim table:

- 128 B → 61.11 MB/s, 256 B → 92.35 MB/s, 512 B → 160.11 MB/s, 1024 B → 167.99 MB/s, 2048 B → 217.86 MB/s, 4096 B → 240.78 MB/s, 8192 B → 317.69 MB/s, 16384 B → 1472.15 MB/s, …, 64 MiB+ → ~150 GB/s ("about 75% of the peak transfer bandwidth of 200GB/s"). github +3 github +2

These are the same numbers the user is seeing within measurement noise. The flat plateau between 512 B and 1024 B (160 → 168 MB/s) and the order-of-magnitude jump between 8 KiB and 16 KiB are signatures of a fixed-cost floor, not of a single threshold being crossed: the 16 KiB jump is the rendezvous pipeline kicking in, not the slow zone "ending."

AMD's separate recommendation on the same page — "Use UCX_RNDV_THRESH environment variable to lower the rendezvous threshold from the 8KB default to 128B to improve the midrange performance" — only moves the cliff to 128 B (producing a sharp dip at 128 B in the user's curve) and does not address the 256 B – 1 KiB band, which is consistent with the mechanism above: lowering `UCX_RNDV_THRESH` changes the *which-path* decision but both candidate paths have a comparable fixed floor in this range. github GitHub

MI250X SDMA engines are tuned for PCIe-Gen4 x16 (~32 GB/s) and cannot saturate xGMI's 200 GB/s; AMD documents this in the "MI200 GPU memory space overview" lab note ("the SDMA engine does not run at this speed, it will not max out the bandwidth of the faster interconnect … bypassing the SDMA engine and replacing it with a type of copy kernel known as a 'blit' kernel … HSA_ENABLE_SDMA=0"). On the LUMI pilot partition (MI250X, ROCm 5.2.3, cray-mpich 8.1.18, May 3 2023, "GPU-Aware MPI with ROCm — EuroCC-AMD Workshop, May 5 2023" deck), intra-OAM 4-link pairs (GCD 0↔1) cap at ~49 GB/s with SDMA and reach ~142 GB/s with blit kernels. This is the *asymptotic* difference; in the 256 B – 1 KiB band both paths pay essentially the same per-call fixed cost — Schieffer et al. (arXiv:2410.00801) state for MI250X: *"memcpy outperforms hipMemcpy for low transfer sizes, up to 512 KB … the measured latency is below 100 ns for transfers up to 16 KB. In contrast, hipMemcpy operations are more complex, as they are delegated to the Heterogeneous System Architecture (HSA) runtime, resulting in higher latency."* MPI cannot use plain `memcpy` between two processes' GCDs; it must go through HSA, so it pays this floor (single-digit μs per call). AMD ROCm + 4

4. Why the Cray PE + Cray MPICH stack has a much milder dip

HPE Cray MPICH on the same node uses the GTL (GPU Transport Layer) and a different small-message path. Per the OLCF tuning deck "Getting the Most out of HPE Cray MPI" (16 Feb 2023), `MPICH_GPU_IPC_THRESHOLD` defaults to 1024 B with the comment: *"This is a performance tunable parameter for controlling when GPU DMA engines are used for intra-node GPU-GPU transfers. Messages smaller than this threshold won't spend time setting up the DMA engine … 1024-byte default."* `MPICH_GPU_EAGER_REGISTER_HOST_MEM` is auto-enabled when GPU support is on (HPE CPE `intro_mpi` manpage): *"registers the CPU-attached shared memory regions with the GPU runtime layers … used for small message intra-node CPU-to-GPU and GPU-to-GPU MPI transfers … amortize the cost of registering memory with the GPU runtime layer."* Together these implement exactly the "small messages skip IPC and use pre-registered host-shared buffers" strategy that UCX in its default Open MPI configuration does not. On Cray MPICH the per-call cost in the 256 B – 1 KiB band is dominated by an already-warm host-staged shared-memory copy rather than by a fresh HSA IPC attach + async-copy round-trip, so the dip is much shallower. This is the most defensible mechanistic explanation for the difference the user observed. Oak Ridge Leadership Com…

5. Topology dependence

Carl Pearson (Sandia National Laboratories), "Interconnect Bandwidth Heterogeneity on AMD MI250x and Infinity Fabric," arXiv:2302.14827 (Feb 28 2023), and Schieffer et al. (arXiv:2410.00801, SC-W '24) both document MI250X's heterogeneous fabric: intra-OAM 4-link (200 GB/s/dir), inter-package 2-link (100 GB/s), inter-package 1-link (50 GB/s), and routed (2-hop) pairs. Neither paper reports a *path-dependent* small-message latency floor: the per-call cost is dominated by host-side HSA runtime work and one async-copy submission, not by wire time, so the 256 B – 1 KiB region is expected to be approximately invariant. What does change across tiers: Hgpu + 3

- the bandwidth asymptote at large sizes (~140 GB/s on quad-link blit, ~75 GB/s on dual-link, ~35 GB/s on single-link, lower on routed pairs);
- the size at which the curve lifts off — slightly later on lower-bandwidth tiers because the fixed cost takes longer to amortise when the per-byte cost is higher;
- on routed pairs, an extra hop adds a second per-call coordination but not a second IPC attach (the IPC handle is direct between source and destination GCD).

The user should therefore expect the 256 B – 1 KiB MB/s values themselves to be almost the same across all four topology tiers, with the curve only diverging from ~16 KiB upward. This is a falsifiable prediction that can be checked with explicit `ROCR_VISIBLE_DEVICES` for representative pairs (e.g. 0–1 intra-OAM, 2–6 inter-package 2-link, 0–4 1-link, 1–4 routed).

Details — Evaluation of the five hypotheses

H1 (`rocm_ipc` IPC-handle setup has a fixed floor): supported. `rocm_ipc` only exposes `put_zcopy` / `get_zcopy` (no AM, no short, no bcopy) per the `ucx_info -d` dump in openucx#6207, and the UCT iface declares `overhead: 400 nsec` plus a `register` cost. Every transfer requires `hsa_amd_ipc_memory_create` on the sender and `hsa_amd_ipc_memory_attach` on the receiver before the actual `hsa_amd_memory_async_copy`; these involve the KFD driver mapping the remote GPU's BAR. Even with the UCX rcache cache, the first hit per peer-buffer pair is expensive, and `osu_bw`'s small-buffer reuse pattern still leaves substantial per-transfer work (lookup, refcount, fence). This explains why lowering `UCX_RNDV_THRESH` to 128 B does not help — rendezvous is then chosen earlier but rendezvous still routes through `rocm_ipc`, which has the floor. github

H2 (bad rendezvous flavour selection): partially supported. UCP protocol selection with `UCX_PROTO_ENABLE=y` (default since UCX 1.16, per UCX-Py docs: *"Its default has been changed to y starting with UCX 1.16.0. The new protocol solves various limitations from the original 'protov1' including, for example, invalid choice of transport in systems with hybrid interconnectivity"*) considers `rndv/get_zcopy`, `rndv/put_zcopy`, `rndv/put_ppln` (pipelined) and `rndv/am` for ROCm device memory; `UCX_RNDV_FRAG_SIZE` defaults to 4 MiB for ROCm (`ucp_rndv_frag_default_sizes[UCS_MEMORY_TYPE_ROCM] = 4 * UCS_MBYTE` in `src/ucp/core/ucp_context.c`). `UCX_RNDV_SCHEME=put_zcopy` vs `get_zcopy` can shift the curve by 10–20 % at large sizes (more relevant for PCIe-attached GPUs than xGMI per AMD's own note: *"UCX v1.12 introduced some new rendezvous protocols that may cause some issues for PCIe (non-XGMI) systems"*), but neither flavour eliminates the 256 B – 1 KiB plateau in any of the published curves. The AMD-recommended PCIe knobs (`UCX_RNDV_PIPELINE_SEND_THRESH=256k`, `UCX_RNDV_FRAG_SIZE=4m`) act on much larger messages. Readthedocs + 2

H3 (HSA/hipMemcpy fixed per-call cost): supported, and probably the dominant cause. Schieffer et al. (2024) explicitly contrast `hipMemcpy` (HSA-mediated, several μs per call for any size up to ~16 KiB) with plain `memcpy` on coherent xGMI mappings (<100 ns up to 16 KiB). UCX in its current form cannot avoid the HSA path for D→D between two processes. `HSA_ENABLE_SDMA=0` swaps SDMA for a blit kernel, which raises asymptotic xGMI bandwidth

from ~50 GB/s to ~140 GB/s on quad-link but adds GPU kernel-launch overhead per transfer — another fixed cost in the same band, so the dip shape is similar with SDMA on or off. Hpc ROCm Blogs

H4 (SDMA-vs-blit crossover boundary): documented at the system level, not at the 256 B – 1 KiB resolution. Public documents do not give a clean sub-KiB SDMA-vs-blit message-size crossover; what they document is the asymptotic difference (~50 vs ~140 GB/s) and that blit kernel-launch overhead is a few μs per call. At 256 B – 1 KiB neither path is asymptotic, so the crossover is irrelevant to the *location* of the dip; it only affects the height of the post-cliff curve at ⩾16 KiB.

H5 (path dependence intra-OAM vs inter-package vs routed): expected to be weak in this band. Per the discussion above, the 256 B – 1 KiB bandwidth should stay near 80–200 MB/s on all tiers; the differentiation appears at large messages.

Documented vs inferred

Claim	Status
`rocm_copy` lacks `am_short`/`am_bcopy` on ROCm device memory	Documented (UCX source `src/uct/rocm/copy/`; `ucx_info -d` in openucx#6207; explicit error "`rocm_copy/rocm_cpy - no am bcopy`") GitHub
`rocm_ipc` lacks AM, only `put_zcopy` / `get_zcopy`	Documented (same dump)
`rocm_copy` iface "latency 10 μs, overhead 0 ns"	Documented for UCX 1.10 (openucx#6207). UCX 1.18 / MI250X / ROCm 6.4.1 number not in any public dump — *inferred* unchanged because hard-coded in `uct_rocm_copy_iface_query`
`hipMemcpy` floor of single-digit μs for ⩽1 KiB on MI250X	Documented (Schieffer et al., arXiv:2410.00801)
Plain `memcpy` across coherent xGMI ≪ 100 ns up to 16 KiB	Documented (Schieffer et al., arXiv:2410.00801)
`MPICH_GPU_IPC_THRESHOLD` = 1024 B default on Cray MPICH	Documented (OLCF "Getting the Most out of HPE Cray MPI," 16 Feb 2023)
`MPICH_GPU_EAGER_REGISTER_HOST_MEM` auto-on with GPU support	Documented (HPE CPE `intro_mpi` manpage)
AMD published `osu_bw` curve shows the same dip	Documented (openucx wiki, "Build and run ROCm UCX OpenMPI")

Claim	Status
256 B – 1 KiB plateau is approximately path-invariant across topology tiers	*Inferred* from mechanism — Schieffer et al. and Pearson 2023 do not break out path-dependence at sub-KiB sizes; **needs empirical confirmation on the LUMI standard-g node**
protov2 (`UCX_PROTO_ENABLE=y`) meaningfully improves the dip on MI250X	Claimed in the UCX 1.16 changelog; not empirically published for MI250X — needs measurement

Recommendations

In order of effort vs expected payoff.

1. **Capture `UCX_PROTO_INFO=y` output on the standard-g node first.** Run `osu_bw` with `UCX_PROTO_INFO=y UCX_LOG_LEVEL=info` and dump the chosen protocol per size band. This pins down whether UCX is in fact selecting `rocm_ipc` rendezvous in the 256 B – 1 KiB region or falling back to a host-staged path. *If the chosen protocol changes when `UCX_RNDV_THRESH` is swept but the bandwidth does not, H3 (HSA fixed cost) is essentially proven for this stack.*
2. **Run a topology sweep on a fixed software stack.** Intra-OAM (0–1), inter-package 2-link (2–6), inter-package 1-link (0–4), routed (1–4), same UCX/OMPI build, plotted on one axis. *If the 256 B – 1 KiB MB/s values differ by less than ~20 % across tiers, the floor is confirmed software-side; if they differ substantially, revisit and consider hardware-link contention.*
3. **Side-by-side with Cray MPICH on the same node.** With `MPICH_ENV_DISPLAY=1 MPICH_VERSION_DISPLAY=1`, sweep `MPICH_GPU_IPC_THRESHOLD` from 128 to 4096 and confirm that the Cray dip moves with the threshold while the UCX dip does not. This is a clean, publishable mechanistic confirmation.
4. **For the EESSI build itself, document but do not "fix" the dip.** Recommend in the thesis that the default EESSI Open MPI / UCX configuration leave `UCX_RNDV_THRESH` at default (let protov2 select) and *not* force it to 128 B globally, because (i) the published 128 B setting introduces a sharp single-point dip at 128 B, (ii) the 256 B – 1 KiB plateau is structural and unchanged either way, and (iii) the 128 B setting penalises large-message tuning on PCIe-attached GPUs that EESSI also supports. Provide users with a module-level recipe (`UCX_RNDV_THRESH`, `UCX_RNDV_SCHEME`, `HSA_ENABLE_SDMA`) and notes pointing to the AMD openucx wiki page and to the LUMI documentation.
5. **(Stretch) File or follow an openucx issue** asking whether `rocm_copy` could expose an `am_bcopy` path using a per-endpoint pre-registered host-pinned staging buffer (effectively the Cray GTL strategy inside UCX). This is the only architectural change that would actually remove the dip on MI250X for UCX users.

Thresholds that would change these recommendations: if `UCX_PROTO_INFO` shows protov2 *already* selects an eager/`rocm_copy` path in the 256 B – 1 KiB band and bandwidth is still flat, no UCX-side tuning will help and step 5 becomes the only real fix. If the dip on the LUMI node turns

out to be *deeper* on routed pairs than on intra-OAM (say >30 % difference), then a per-link cost component exists beyond the HSA floor and the mechanism in §5 needs revision.

Caveats

- The exact `latency`/`overhead` fields reported by `ucx_info -d` for `rocm_copy` and `rocm_ipc` on UCX 1.18 against ROCm 6.4.1 / MI250X have not been verified in public documentation; the values cited above are from a UCX 1.10 dump on an MI60 node (openucx#6207). Capability flags in `src/uct/rocm/copy/rocm_copy_iface.c` and `src/uct/rocm/ipc/rocm_ipc_iface.c` are essentially unchanged between those versions, but the user should capture a fresh `ucx_info -d` on the standard-g node and quote those numbers in the thesis.
- The mechanism proposed for the milder Cray MPICH dip is inferred from documented variables and tuning slides rather than from a controlled experiment. A single back-to-back `osu_bw` run on the same node with both stacks and an `MPICH_GPU_IPC_THRESHOLD` sweep would strengthen the thesis.
- The claim that the slow zone is approximately topology-invariant on MI250X is a mechanistic prediction. Neither Schieffer et al. (2024) nor Pearson (2023) publish a topology-by-topology breakdown of sub-KiB `osu_bw` bandwidth, and De Sensi et al. (SC24) report node-level metrics rather than per-pair-tier small-message curves on LUMI. Empirical confirmation is required. arxiv
- All publicly published MI250X `osu_bw` curves the report relies on (openucx wiki, AMD GPUOpen lab notes, CASTIEL/LUMI EuroCC deck) used ROCm 5.2.3–6.1.0 and UCX 1.14–1.16; the user's stack is ROCm 6.4.1 + UCX 1.18, so absolute MB/s values may differ by 10–30 %. The *shape* of the dip and the *location* of the 256 B – 1 KiB band are expected to be unchanged because they are determined by the HSA runtime and the UCT capability model, not by ROCm minor-version performance.
- `UCX_PROTO_ENABLE` should be confirmed on with `ucx_info -c | grep PROTO_ENABLE` in the EESSI build; protov1 makes worse path choices on hybrid-topology GPU systems per the UCX 1.16 changelog.